



Reflection and Traceability Report on RoCam

Team #3, SpaceY

Zifan Si

Jianqing Liu

Mike Chen

Xiaotian Lou

1 Symbols, Abbreviations, and Acronyms

Symbol	Description
CI	Continuous Integration
CV	Computer Vision
DDP	Distributed Data Parallel
FR	Functional Requirement
HA	Hazard Analysis
IPC	Inter-Process Communication
MG	Module Guide
MIS	Module Interface Specification
NFR	Non-Functional Requirement
PCB	Printed Circuit Board
PID	Proportional-Integral-Derivative
PR	Pull Request
SRS	Software Requirements Specification
SSE	Server-Sent Events
TA	Teaching Assistant
VnV	Verification and Validation
YOLO	You Only Look Once

Table 1: Symbols, abbreviations, and acronyms used in this report.

See SRS documentation [\[1\]](#) for the complete table of symbols and units used in RoCam.

Contents

1	Symbols, Abbreviations, and Acronyms	2
2	Changes in Response to Feedback	4
2.1	SRS and Hazard Analysis	5
2.2	Design and Design Documentation	7
2.3	VnV Plan and Report	8
3	Extras	9
4	Design Iteration (LO11 — PrototypeIterate)	9
5	Design Decisions (LO12)	14
6	Economic Considerations (LO23)	14
7	Reflection on Project Management (LO24)	15
7.1	How Does Your Project Management Compare to Your Development Plan	15
7.2	What Went Well?	15
7.3	What Went Wrong?	16
7.4	What Would you Do Differently Next Time?	16
8	Ethics and Equity (LO18)	16
9	Reflection on Capstone	17
9.1	Which Courses Were Relevant	17
9.2	Knowledge/Skills Outside of Courses	17

List of Tables

1	Symbols, abbreviations, and acronyms used in this report.	2
2	Peer Review Items Not Implemented	4
3	SRS Feedback Tracking and Implementation	6
4	Hazard Analysis Feedback Tracking and Implementation	6
5	MG Documentation Feedback Tracking and Implementation	7
6	MIS Documentation Feedback Tracking and Implementation	7
7	VnV Plan Feedback Tracking and Implementation	8
8	VnV Report Feedback Tracking and Implementation	8

List of Figures

1	Revision 0 operator interface before the final single-page layout and integrated hardware presentation.	10
2	Initial hardware revision used during early integration and control experiments.	11
3	Final RoCam hardware and operator interface at the McMaster Expo on April 7, 2026 (view 1).	12
4	Final RoCam hardware and operator interface at the McMaster Expo on April 7, 2026 (view 2).	13

2 Changes in Response to Feedback

This section summarizes the changes made to project deliverables in response to feedback from the TA, [Tanya Djavaheerpour](#), peer reviewers, the course staff, the project supervisor ([Dr. Shahin Sirouspour](#)), and the client ([McMaster Rocketry Team](#)). Our team used GitHub issues, pull requests, and follow-up commits to keep the feedback traceable.

All TA and supervisor suggestions were implemented in the final revision of the documentation. Most peer review suggestions were also implemented. Several peer review items were intentionally not adopted because they were duplicates, conflicted with the structure we selected for the document, or were outside the intended scope. In those cases, we replied in the corresponding GitHub issues with the rationale for not implementing the suggestion.

Table 2: Peer Review Items Not Implemented

Issue	Feedback Summary	Reason Not Implemented
Issue #185	Design Doc - Minimal Anticipated changes	We kept the anticipated changes section focused on the most realistic high-impact changes. Adding more speculative items would have reduced clarity without improving the design rationale.
Issue #184	Design Doc - Clarify and tighten module dependency	We did not change the dependency structure because the existing relationships already matched the implemented architecture. Tightening them further would have hidden valid interactions between modules.
Issue #133	V&V Plan: include structured review procedure for 3.3 Design Verification	We kept the review procedure lightweight because roles, review artifacts, and verification activities were already documented elsewhere in the plan. Adding another procedure would have duplicated process information.
Issue #116	Lack of Traceability Matrix for Requirements	We did not add a separate matrix in response to this issue because the final document already provided sufficient traceability through the existing requirement mappings and linked documentation. A second overlapping matrix would have been redundant.
Issue #112	Lack of Identification of Template used	We did not add a separate template-identification note because the document had already diverged substantially from the original capstone template and the final deliverable was meant to stand on its own structure and content.
Continued on next page		

Table 2 continued from previous page

Issue	Feedback Summary	Reason Not Implemented
Issue #69	Did not assign roles to individual members and did not create a plan for rotating roles	We intentionally kept stable team roles instead of rotating them because the project depended on specialized ownership in hardware, CV, backend, and frontend work. Forced rotation would have increased coordination cost without clear benefit.
Issue #181	Design Doc - Add leaf modules to API gateway	We did not implement this issue separately because it was marked as a duplicate of an existing feedback item. The underlying concern was already tracked elsewhere, so keeping both would have duplicated work and discussion.
Issue #372	No clear differentiation in Section 8	This comment arrived late and remained open at the time of writing. We did not revise the document again before submission, but the issue thread records the outstanding concern.
Other peer review tickets	Additional peer review issues in the repository that are not listed above	Some review activity was conducted jointly with two teams at the same time. As a result, a few peer review tickets in the repository were associated with the combined review process rather than being direct feedback that applied to this document set, so they are not counted here as unimplemented RoCam feedback.

The main documentation revision was completed in [Commit 30cd72c](#) through [PR #365](#), with an additional VnV Report update in [Commit 45a7df6](#). Where a table row does not list a separate GitHub issue, the change came from direct review annotations or discussion during the corresponding document review.

The tables below focus on feedback items that produced document changes. Peer review issues that were closed as WONTFIX, duplicate, stale, or otherwise did not produce a document change are accounted for in their linked issue threads rather than repeated as change rows in the implementation tables below.

Supervisor feedback had the largest impact on safety-related documentation. In particular, the request for a clearer emergency-stop strategy directly led to the addition of SR4 in the Hazard Analysis [2] and the corresponding traceability updates in the SRS [1].

2.1 SRS and Hazard Analysis

Table 3 outlines the changes for the SRS document. Table 4 outlines the changes for the Hazard Analysis document.

Table 3: SRS Feedback Tracking and Implementation

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
TA Tanya	Typo: “equipments”.	Corrected the wording to “equipment”.	Commit 30cd72c Issue #107
TA Tanya	Typo: “Through out”.	Corrected the wording to “Throughout”.	Commit 30cd72c Issue #107
TA Tanya	“500CAD” missing spacing.	Reformatted the value as “500 CAD”.	Commit 30cd72c Issue #107
TA Tanya	Duplicate requirement ID AR-1.	Renamed the duplicate entry to ACR-1 to remove ambiguity.	Commit 30cd72c Issue #107
TA Tanya	Missing cross-references to the Hazard Analysis.	Added explicit HA references throughout the SRS traceability material.	Commit 30cd72c Issue #107
Peer	Phase-in priorities were unclear.	Clarified the priority levels used across requirement categories.	Commit 30cd72c

Table 4: Hazard Analysis Feedback Tracking and Implementation

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
TA Tanya	Placeholder “Date1/Date2” remained in the revision table.	Replaced the placeholder values with the actual revision dates.	Commit 30cd72c Issue #107
TA Tanya	Typo: “a existing”.	Corrected the phrase to “an existing”.	Commit 30cd72c Issue #107
TA Tanya	Typo: “moodule”.	Corrected the spelling to “module”.	Commit 30cd72c Issue #107
TA Tanya	Typo: “a off the shelf”.	Corrected the phrase to “an off-the-shelf”.	Commit 30cd72c Issue #107
TA Tanya	Typo: “accerlerated”.	Corrected the spelling to “accelerated”.	Commit 30cd72c Issue #107
TA Tanya	Typo: “excesses”.	Corrected the wording to “exceeds”.	Commit 30cd72c Issue #107
TA Tanya	Inconsistent wording around “off-the-shelf” terminology.	Standardized the term and capitalization across the document.	Commit 30cd72c Issue #107
TA Tanya	SR1–SR3 lacked sufficient detail.	Expanded the requirements with clearer conditions, thresholds, and verification intent.	Commit 30cd72c Issue #107
Continued on next page			

Table 4 continued from previous page

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
Supervisor	No emergency-stop mechanism was defined.	Added SR4 (Emergency Stop) with both hardware and software triggers.	Commit 30cd72c
TA Tanya	Missing traceability between HA and SRS.	Added an HA-to-SRS traceability table mapping hazards to safety requirements.	Commit 30cd72c Issue #107

2.2 Design and Design Documentation

Table 5 outlines the feedback incorporated into the Module Guide [3]. Table 6 outlines the feedback incorporated into the Module Interface Specification [4].

Table 5: MG Documentation Feedback Tracking and Implementation

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
TA Tanya	Remaining capstone template comments.	Removed all remaining template comments from the MG.	Commit 30cd72c Issue #164 Issue #240
TA Tanya	Minor typos throughout the document.	Corrected the identified spelling and grammar issues.	Commit 30cd72c Issue #164 Issue #240
TA Tanya	Module hierarchy diagram was hard to read.	Improved formatting and label placement in the hierarchy diagram.	Commit 30cd72c Issue #164 Issue #240

Table 6: MIS Documentation Feedback Tracking and Implementation

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
TA Tanya	Year values still referred to 2025 instead of 2026.	Corrected the year references throughout the MIS.	Commit 30cd72c Issue #164 Issue #240
TA Tanya	Remaining capstone template comments.	Removed the leftover template comments from the MIS.	Commit 30cd72c Issue #164 Issue #240
TA Tanya	Typo: “presistent”.	Corrected the spelling to “persistent”.	Commit 30cd72c Issue #164 Issue #240

2.3 VnV Plan and Report

Table 7 outlines the changes for the VnV Plan [5] document. Table 8 outlines the changes for the VnV Report [6] document.

Table 7: VnV Plan Feedback Tracking and Implementation

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
TA Tanya	Remaining capstone template comments.	Removed all remaining template comments from the VnV Plan.	Commit 30cd72c Issue #122
TA Tanya	Unit test descriptions were too vague.	Added expected inputs, outputs, and pass criteria to the affected tests.	Commit 30cd72c Issue #122
TA Tanya	Missing unit test summary section.	Added a Unit Testing Scope section summarizing coverage and strategy.	Commit 30cd72c Issue #122
Peer	Traceability matrix was incomplete.	Expanded the matrix to cover all FR and NFR mappings.	Commit 30cd72c Issue #342
Peer	Missing non-functional test details.	Added performance, usability, and stress-test specifications.	Commit 30cd72c Issue #343
Peer	Pass indicators were not colour-blind safe.	Changed the visual indicator to teal text and kept a textual status label.	Commit 30cd72c Issue #346

Table 8: VnV Report Feedback Tracking and Implementation

Feedback Source	Feedback Summary	Changes Made	Related Commits/Issues
TA Tanya	Report lacked scope clarification.	Added an introductory paragraph clarifying the report scope.	Commit 45a7df6 Issue #337
TA Tanya	“Changes due to testing” section was incomplete.	Added summary tables linking V&V evidence to quality improvements.	Commit 45a7df6 Issue #337
Peer	Missing CI evidence.	Added an Automated Testing section referencing GitHub Actions workflows.	Commit 45a7df6 Issue #148
Peer	Future work was not discussed.	Added a Future Testing Improvements subsection.	Commit 45a7df6 Issue #149

3 Extras

Our team completed two supplementary reports that were approved by the course instructor as “Extras” for the capstone project.

1. **Circuit Design Report** (docs/Extras/CircuitDesign/, [7]): Documents the design of the custom STM32-based PCB for gimbal motor control. Covers schematic design, PCB layout, component selection, and manufacturing. The report includes the full schematic, board layout, and photographs of the assembled hardware. This extra is deeply integrated with the project—the custom PCB is the physical interface between the Jetson Orin Nano and the gimbal motors.
2. **Machine Learning Report** (docs/Extras/MachineLearning/, [8]): Documents the multi-stage YOLO training pipeline for rocket detection. Covers dataset preparation, augmentation strategy, three-stage progressive training, TensorRT optimization, and accuracy benchmarking. The report includes confusion matrices, precision-recall curves, and training loss graphs across seven training iterations. This extra represents the core intelligence of the system—the CV model that enables automatic rocket tracking.

4 Design Iteration (LO11 — PrototypeIterate)

Our design evolved significantly from Revision 0 to Revision 1, driven by real-world testing, supervisor feedback, and performance measurements.

User Interface Evolution

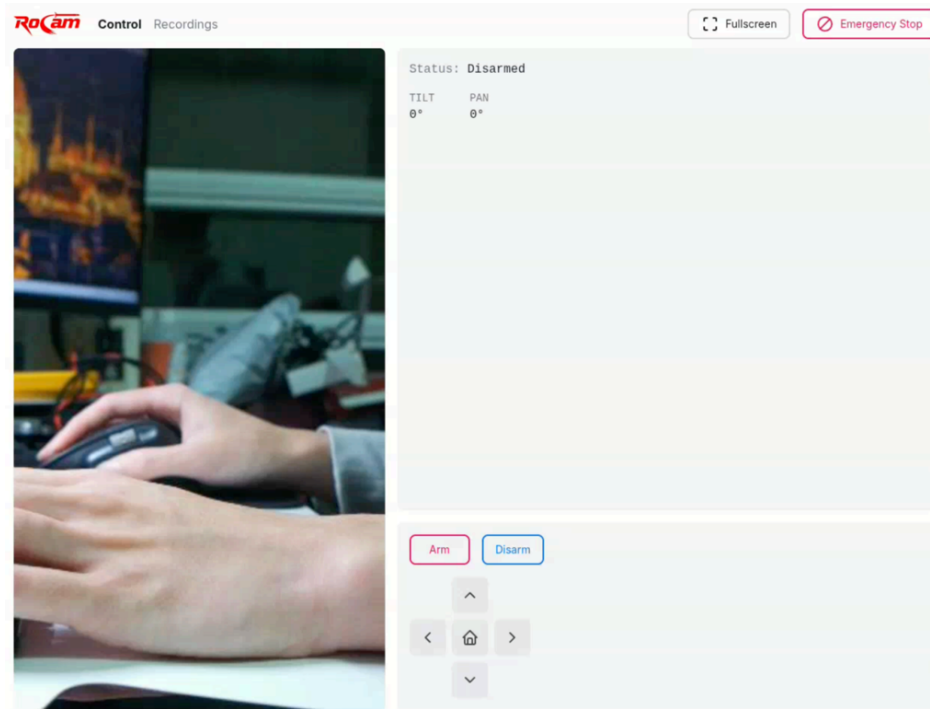


Figure 1: Revision 0 operator interface before the final single-page layout and integrated hardware presentation.

- **Rev 0:** The initial UI was a multi-page web application with separate views for video, controls, and logs (Figure 1). Operators found switching between pages disorienting in outdoor settings.
- **Rev 1:** After usability feedback, we redesigned the interface as a single-page application integrated into the final deployed system (Figures 3 and 4) with all critical controls visible simultaneously. The bilingual (English/French) support was added to meet Canada's official bilingualism requirements.

Hardware Evolution

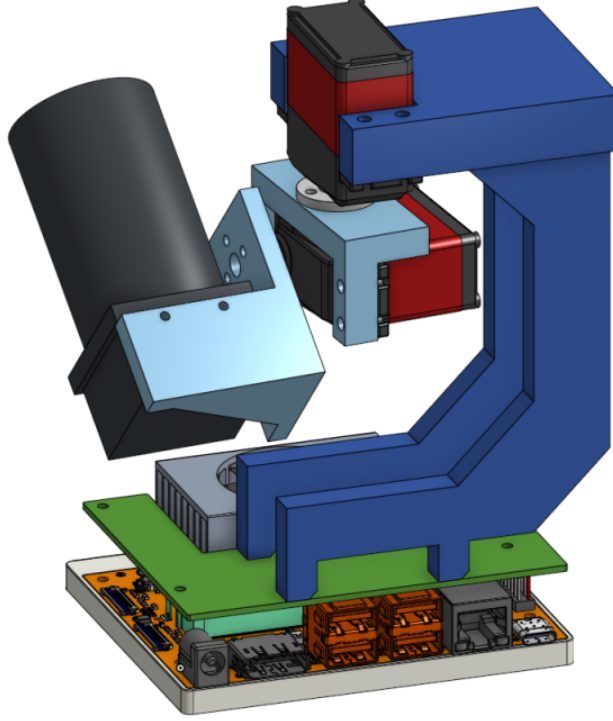


Figure 2: Initial hardware revision used during early integration and control experiments.

- **Rev 0:** The initial prototype used an off-the-shelf servo controller connected to the Jetson via USB-serial (Figure 2). The gimbal response was sluggish (>300 ms latency) and lacked encoder feedback, causing drift during tracking.
- **Rev 1:** We designed a custom STM32-based PCB and integrated it into the final RoCam system shown in Figures 3 and 4, with a tuned PID control loop and direct encoder integration. This reduced gimbal command latency to <20 ms and eliminated drift. The decision was driven by measured performance gaps during Rev 0 field testing.

Final Integrated System

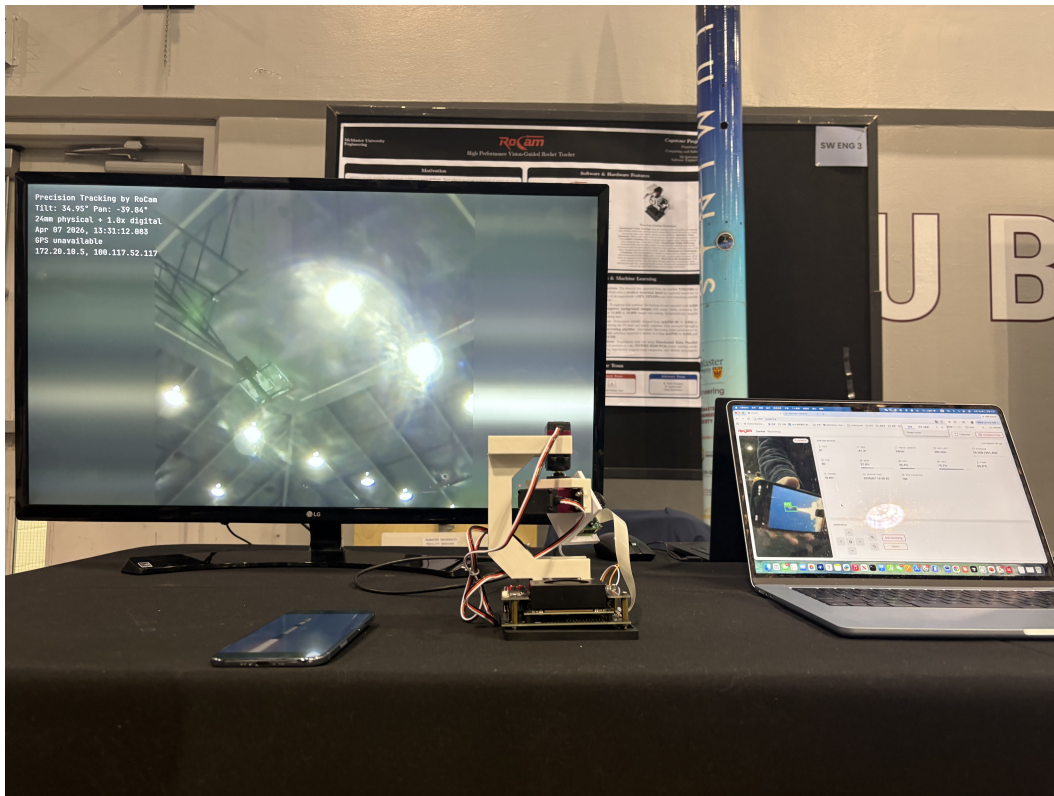


Figure 3: Final RoCam hardware and operator interface at the McMaster Expo on April 7, 2026 (view 1).



Figure 4: Final RoCam hardware and operator interface at the McMaster Expo on April 7, 2026 (view 2).

Software Architecture Evolution

- **Rev 0:** A monolithic Python process handled CV inference, livestream encoding, and API serving. Frame rate dropped below 15 fps under load.
- **Rev 1:** We separated the system into four independent processes—CV, Livestream, Transcode, and Control—communicating via shared-memory IPC. This achieved 60 fps at 1080p with <120 ms end-to-end latency.

Machine Learning Pipeline Evolution

- **Rev 0:** Single-stage YOLO training on a small dataset (train63, mAP50 = 0.887). The model struggled with small rockets at long range and rockets against cluttered backgrounds.
- **Rev 1:** A three-stage progressive training pipeline (Stage 1: 300 epochs base training → Stage 2: robustness fine-tuning → Stage 3: polishing) with copy-paste augmentation for small targets. Final model (V3-Phase2) achieved mAP50 = 0.948 and mAP50-95 = 0.651, a 7% improvement in mAP50 over Rev 0.

5 Design Decisions (LO12)

Limitations

- **Jetson Orin Nano compute budget:** The 40 TOPS INT8 performance ceiling constrained our choice of detection model. Larger YOLO variants (e.g., YOLO-L, YOLO-X) exceeded the real-time inference budget. We selected YOLO26s-P2, which achieves 60 fps after TensorRT FP16 optimization while maintaining competitive accuracy.
- **Single camera sensor:** The system uses a single camera, which limits depth estimation. We mitigated this by using focal-length-based apparent size as a proxy for range, accepting reduced accuracy at extreme distances (>500 m).
- **Wi-Fi range:** The operator interface communicates over Wi-Fi, limiting the practical operating range to approximately 100 m line-of-sight. This was acceptable for the target use case (collegiate launch events) but would not scale to high-altitude or remote launches.

Assumptions

- **Single-target tracking:** We assumed one rocket would be in the field of view at a time. Multi-target scenarios (e.g., cluster launches) are not supported.
- **Daylight operation:** The YOLO model was trained exclusively on daylight imagery. Night launches or low-light conditions are outside the tested operating envelope.
- **Operator presence:** We assumed a human operator would always be present to initiate tracking and intervene if needed, justifying the semi-automatic design rather than a fully autonomous system.

Constraints and Key Decisions

- **Process separation (CV, Livestream, Transcode):** The real-time performance requirement (1080p at 60 fps with <120 ms latency) could not be met with a single-process architecture. We separated the CV inference, livestream encoding, and transcoding into independent worker processes communicating via shared memory and IPC. This decision was driven by measured performance bottlenecks during Rev 0 testing.
- **Flask + SSE over WebSocket:** We chose Flask with Server-Sent Events rather than WebSocket. SSE is simpler and sufficient for our use case, which requires only unidirectional status updates. Bidirectional commands are handled via standard HTTP POST requests.
- **Custom PCB over off-the-shelf servo controller:** The gimbal requires precise two-axis control with real-time encoder feedback at rates that commodity controllers could not reliably provide. The custom STM32-based PCB achieves smoother, more responsive gimbal movement.
- **React/TypeScript frontend:** We chose React with TypeScript for type safety and responsive single-page design optimized for outdoor tablet use.

6 Economic Considerations (LO23)

Market analysis: There is currently no commercially available product at a comparable price point that provides automated rocket tracking with live video feed for amateur and collegiate rocketry. Existing solutions either require manual operation (standard tripod-mounted cameras) or cost upwards of \$5,000 (commercial tracking mounts). The model rocketry community in North America

includes approximately 50,000 active hobbyists and over 200 university rocketry teams, representing a meaningful niche market.

Cost to produce: Our current hardware bill of materials is approximately 350 CAD, covering the camera sensor, gimbal assembly, Jetson Orin Nano, custom PCB components, and structural hardware. For a production version, additional costs would include a weatherproof enclosure (~50 CAD), a user-friendly setup wizard (software development), and regulatory compliance testing (e.g., FCC for the Wi-Fi module). We estimate the total production cost at approximately 500–600 CAD per unit at low volumes.

Pricing and break-even: We estimate a retail price of 800–1,200 CAD would be competitive. At an average margin of ~350 CAD per unit, approximately 500 units would need to be sold to recover tooling, injection mould, and certification costs (estimated at ~175,000 CAD).

Go-to-market strategy: Given the niche market, an open-source strategy would be most effective:

- Selling pre-assembled hardware kits for users who prefer not to build their own.
- Offering premium support and custom integration services for university teams.
- Partnering with rocketry suppliers for distribution.

7 Reflection on Project Management (LO24)

7.1 How Does Your Project Management Compare to Your Development Plan

We largely followed our Development Plan. Weekly team meetings were held consistently, with agendas and minutes tracked via GitHub Issues. Asynchronous communication was conducted on Discord, and bi-weekly meetings with our supervisor, Dr. Sirouspour, were held over Zoom. Team member roles aligned with the plan: Zifan Si led frontend development, Jianqing Liu handled backend architecture and served as team lead, Mike Chen was responsible for the computer vision pipeline, and Xiaotian Lou managed hardware integration and PCB design.

We used GitHub Projects for task tracking, with a Kanban-style board organized by sprint. All code changes went through pull request review before merging. The technology stack matched our plan: Python (Flask) backend, React/TypeScript frontend, YOLO with TensorRT for CV, and a custom STM32 PCB for gimbal control, all deployed on the Jetson Orin Nano.

7.2 What Went Well?

- **CI/CD pipeline:** GitHub Actions for automated linting and testing caught issues early and maintained code quality. Every PR was required to pass CI checks before merging.
- **Structured sprint process:** Two-week sprints with clear deliverables kept the team on track and made progress visible.
- **PR-based code review:** Requiring at least one approving review before merging improved code quality and knowledge sharing. Several bugs were caught during review.
- **Issue-driven feedback tracking:** Using GitHub Issues to track all feedback items ensured every piece of feedback was addressed and traceable.

7.3 What Went Wrong?

- **Camera sensor change:** A mid-project switch to a different camera sensor required significant rework of the CAD model, updated calibration parameters, and revised latency measurements (~2 weeks unplanned effort).
- **Integration complexity underestimated:** Integrating the custom PCB with the Jetson, the CV pipeline with the livestream, and the frontend with the backend proved more complex than anticipated. Serial communication timing required substantial debugging.
- **Limited field testing:** Due to weather constraints and schedule pressure, we conducted fewer outdoor field tests than planned.

7.4 What Would you Do Differently Next Time?

- **Start hardware integration earlier:** Begin hardware-software integration testing in the first month to surface issues sooner.
- **Allocate more time for field testing:** Schedule regular outdoor testing sessions starting from Rev 0.
- **Component sourcing contingency:** Identify backup components for critical hardware (especially sensors) at project outset.
- **Documentation-as-you-go:** Maintain documents in parallel with development to distribute workload more evenly.

8 Ethics and Equity (LO18)

Our team applied principles of equity and universal design throughout the project:

- **Bilingual user interface:** The operator interface supports both English and French, reflecting Canada’s official bilingualism and ensuring accessibility for users comfortable in either language.
- **Colour-blind safe indicators:** After peer feedback ([Issue #346](#)), we replaced all green status indicators with teal ([Pass](#)), achieving a contrast ratio of at least 4.5:1 (WCAG AA compliance). All status information is also conveyed through text labels, not colour alone.
- **Outdoor-readable single-page UI:** Large, high-contrast elements optimized for tablets in bright outdoor conditions. Operators wearing sunglasses or gloves can still interact effectively.
- **Confirmation prompts for dangerous actions:** All potentially dangerous operations (gimbal reset, emergency stop override) require explicit confirmation, protecting both the operator and bystanders.
- **Open-source licensing:** Released under the MIT license, ensuring equitable access regardless of institutional affiliation or budget.
- **Privacy considerations:** The YOLO model was trained exclusively on rocket imagery with no person-detection capabilities. The system operates on a local network with no cloud data transmission, minimizing privacy risks at launch events.

9 Reflection on Capstone

9.1 Which Courses Were Relevant

- **SFWRENG 3A04 — Software Design III:** Applied to modular decomposition, interface design, and the overall software structure in the MG and MIS.
- **SFWRENG 3BB4 — Software Design II:** Helpful for state management, abstraction boundaries, and designing maintainable component interactions.
- **SFWRENG 3DX4 — Dynamic Systems and Control:** Directly relevant to the gimbal dynamics, feedback control behaviour, and PID tuning considerations.
- **SFWRENG 3MX3 — Signals and Systems:** Provided the signal processing background needed for camera motion response, filtering, and reasoning about time-varying behaviour in the control path.
- **SFWRENG 3RA3 — Software Requirements & Security Concepts:** Applied throughout the SRS, hazard traceability, safety thinking, and security-related requirement specification.
- **SFWRENG 3S03 — Software Testing:** Directly supported the VnV Plan, VnV Report, unit testing strategy, and traceability from requirements to verification evidence.
- **ENGINEER 3PX3 — Engineering Economics:** Used for the market, pricing, cost, and break-even analysis in the economic considerations section.
- **COMPSCI 3DM3 — Introduction to Data Mining:** Helpful for dataset handling, evaluation metrics, and analysing model performance trends during YOLO training.
- **COMPSCI 4AL3 — Applications of Machine Learning:** Directly relevant to the object detection pipeline, model training iterations, and performance evaluation used in the CV subsystem.

9.2 Knowledge/Skills Outside of Courses

- **TensorRT optimization:** Converting YOLO models to TensorRT engines, INT8 calibration, and inference benchmarking on the Jetson.
- **GStreamer pipelines:** Low-latency video encoding/decoding with hardware-accelerated codecs (NVENC/NVDEC).
- **PCB design:** Schematic capture, PCB layout, DRC, and manufacturing (KiCad, EMI/EMC considerations for motor control).
- **Real-time systems programming:** Process isolation, shared memory IPC, and Linux real-time scheduling for <120 ms latency.
- **React and TypeScript:** Component-based UI architecture, state management, and responsive design for the operator interface.
- **Embedded firmware:** STM32 firmware with UART protocols and PID motor control (Embassy async runtime in Rust).

References

- [1] Vision-Guided Tracker Team, “Software requirements specification,” 2025, project document, this repository.
- [2] —, “Hazard analysis,” 2026, project document, this repository.
- [3] —, “Software architecture (module guide),” 2025, project document, this repository.
- [4] —, “Software detailed design (module interface specification),” 2025, project document, this repository.
- [5] —, “Verification and validation plan,” 2025, project document, this repository.
- [6] —, “Verification and validation report,” 2026, project document, this repository.
- [7] —, “Circuit design report,” 2026, project document, this repository.
- [8] —, “Machine learning report,” 2026, project document, this repository.